Chess Next-Board Generation via Energy-Guided Diffusion: Project Specification

Project Overview

This project trains a generative model that, given a chess position (the "problem board"), produces the chess position that results from the best next move (the "trace board"). The system is JEPA-inspired in its use of a joint-embedding architecture with a target encoder, and combines a denoising diffusion model with an energy-based model (EBM) that guides diffusion sampling via gradient-based refinement.

The core inference loop interleaves diffusion denoising steps with energy-gradient refinement steps, so the diffusion model proposes increasingly clean boards while the energy model corrects each proposal toward configurations it considers high-quality. After all steps, a softmax over piece channels produces a discrete legal board.

The dataset is 180k (problem FEN, best-move trace FEN) pairs generated by Stockfish, with no auxiliary information (no eval scores, no continuations beyond one ply). Training compute target is a single RTX 4070 Ti Super.

Implementation Environment: Marimo Notebooks

This project is implemented entirely in marimo notebooks, not as a conventional Python package. Every module described in the file layout below is a `.py` file in marimo's notebook format, not a standard Python module. A code-generating model producing files for this project must produce marimo notebook `.py` files, not regular Python scripts.

What marimo files look like

A marimo notebook is a `.py` file where the entire notebook is wrapped in a `marimo.App` and each cell is a function decorated with `@app.cell`. The format is executable Python (you can run it directly with `python notebook.py`) and also opens as an interactive reactive notebook with `marimo edit notebook.py`.

A minimal marimo file has this structure:

python

```python
import marimo

__generated_with = "0.9.0"
app = marimo.App(width="medium")


@app.cell
def __():
    import marimo as mo
    return (mo,)


@app.cell
def __():
    import torch
    import torch.nn as nn
    return nn, torch


@app.cell
def __(torch):
    x = torch.randn(4, 18, 8, 8)
    return (x,)


@app.cell
def __(mo):
    mo.md("## Some explanatory header")
    return


@app.cell
def __(nn, torch, x):
    layer = nn.Conv2d(18, 64, 3, padding=1)
    y = layer(x)
    print(y.shape)
    return layer, y


if __name__ == "__main__":
    app.run()
```

Marimo cell rules a code generator must follow

- **Each cell is a function named `__` decorated with `@app.cell`.** The function body is the cell's code.
- **Every variable a cell uses must be in its argument list, and every variable a cell defines and is used downstream must be in its return tuple.** Marimo statically analyzes these to build the reactive dependency graph. A cell that needs `torch` and `x` is `def __(torch, x):`. A cell that defines `model` and `optimizer` ends with `return model, optimizer`. Single-element returns use the trailing-comma tuple syntax: `return (mo,)`.
- **No global mutation across cells.** Each cell's outputs are immutable from the perspective of other cells; downstream cells consume them as inputs. Reassigning a name from a different cell is a marimo error.
- **No top-level code outside cells**, except the `import marimo`, the `__generated_with` string, the `app = marimo.App(...)` line, and the `if __name__ == "__main__": app.run()` footer.
- **Markdown cells use `mo.md("...")`.** Use them generously — every section heading and every block of explanation in a notebook should be a markdown cell, not a Python comment, so the rendered notebook reads top-to-bottom as documentation.
- **One concept per cell.** Cells should be small. Don't pack a whole training loop into one cell — split into "build model," "build optimizer," "train one epoch," etc., each in its own cell.
- **Imports are cells too.** Group related imports in a single cell at the top of each notebook; downstream cells take those module names as arguments.

File layout under marimo

The project uses a **four-folder structure**. Three of the folders contain marimo notebook `.py` files (the dataloader-and-architecture folder, the training folder, and the inference folder). The fourth folder holds saved model weights and checkpoints. There is no separate plain-Python `lib/` directory — all code lives inside marimo notebooks.

```
chess_ebm/
├── core/                       # dataloader + all model architectures
|   ├── dataset.py              # marimo notebook: BoardDataset (single boards) +
ChessPairDataset (3-tuple) + corruption strategies + FEN↔tensor
|   ├── vqvae.py                # marimo notebook: full VQ-VAE built on
diffusers.VQModel + reconstruction loss helpers
|   ├── encoder.py              # marimo notebook: encoder extraction from VQ-VAE
checkpoint, freezing logic, EMA utility
|   ├── energy.py               # marimo notebook: energy model class (online + EMA
target + fusion head + 4-way concat)
|   └── diffusion.py            # marimo notebook: diffusers.UNet2DModel wrapper +
DDPM/DDIM scheduler setup + legality loss
├── training/                   # one marimo notebook per training stage
|   ├── stage0_vqvae.py         # stage 0: VQ-VAE pretraining on problem boards
only
|   ├── stage1_energy.py        # stage 1: energy pretraining (easy mix)
|   ├── stage2_diffusion.py     # stage 2: diffusion pretraining (easy mix) + end-
of-stage legality gate
|   ├── stage3_joint.py         # stage 3: concurrent training (easy mix), optional
legality loss
|   └── stage4_hard.py          # stage 4: concurrent training (hard mix, lower LR)
├── inference/                  # marimo notebook for the full inference loop
|   └── sample.py               # diffusion + energy refinement loop,
discretization, eval helpers
└── checkpoints/                # all saved model weights and run artifacts
    ├── vqvae.pt                # output of stage 0 (full VQ-VAE; only the encoder
+ quant_conv are used downstream)
    ├── stage1_energy.pt        # output of stage 1
    ├── stage2_diffusion.pt     # output of stage 2
    ├── stage3_joint.pt         # output of stage 3
    └── stage4_final.pt         # output of stage 4 (the deployed model)
```

How notebooks share code without a lib/ folder

Because every code file is a marimo notebook and marimo notebooks are not importable as plain Python modules, sharing code across notebooks needs a concrete convention. There are two viable patterns; **this project uses Pattern A**:

Pattern A (this project): import marimo notebooks as modules using marimo's notebook-as-module support.

Marimo notebooks can be imported as Python modules — when you `import core.energy`, marimo executes the notebook's cells in dependency order and exposes the names defined at the notebook level (class definitions, function definitions, top-level constants) as module attributes. Side-effectful cells (training loops, plots) are still executed on import, so `core/` notebooks must be **definition-only**: they may define classes, functions, and constants, but they must not run training or any heavy work at the top level. Any sanity-check or visualization cells in `core/` notebooks should be guarded so they only run when the notebook is opened in `marimo edit`, not on import:

```python
@app.cell
def __(mo):
    # Only show this when editing the notebook, not when importing it
    mo.md("### Sanity check (only renders in marimo edit)")
    return
```

Training and inference notebooks then do `from core.dataset import ChessDataset`, `from core.energy import EnergyModel`, etc.

Pattern B (not used here, mentioned for context): keep a separate plain-Python `lib/` folder. We are explicitly not doing this — the user's preferred structure is the four folders above.

Folder responsibilities

- `core/` — every model architecture and the dataloader. Each file in this folder is a marimo notebook that, when opened with `marimo edit`, lets you inspect and unit-test that component (run a forward pass, visualize a batch, etc.) interactively. When imported by the training or inference notebooks, only the class and function definitions are picked up. Nothing in `core/` performs training or saves checkpoints.
- `training/` — one marimo notebook per training stage. Each notebook imports from `core/`, builds the relevant models, runs the stage's training loop, and writes a checkpoint to `checkpoints/`. Each stage notebook *reads* the previous stage's checkpoint and *writes* its own.
- `inference/` — a single marimo notebook that loads the final checkpoint, runs the diffusion + energy refinement loop on validation problems, performs discretization, and computes the trace-match and piece-wise accuracy metrics. Evaluation is folded into this notebook rather than being separate, since both loading the final model and running it are the same setup.
- `checkpoints/` — disk artifacts only. No code. Holds the input VQ-VAE encoder weights and every output checkpoint produced by the training stages.

What's inside each `core/` notebook

Each `core/` notebook should be organized as:

1. Markdown cell describing what the notebook defines.
2. Imports cell.
3. One cell per class definition or major function definition (helper functions can be grouped).
4. Optional sanity-check cells at the bottom that exercise the definitions — these may print shapes, render small visualizations, or run a tiny forward pass. They will execute when the notebook is imported, so keep them cheap and side-effect-free (no file writes, no training).

For example, `core/dataset.py` defines: FEN↔tensor conversion functions, the three corruption strategy functions, the `ChessDataset` class (taking `corruption_mix` as a constructor argument), and a `make_dataloader(...)` helper. Its sanity-check cells construct a tiny dataset, pull one batch, and print tensor shapes.

What's inside each `training/` notebook

Each training notebook should be organized as:

1. Markdown cell stating the stage and its goal.
2. Imports cell, including `from core.dataset import ...`, `from core.energy import ...`, etc.
3. Cell to load the previous stage's checkpoint:
   - Stage 0: no previous checkpoint; build VQ-VAE from scratch.
   - Stage 1: load `checkpoints/vqvae.pt`, extract encoder weights into both online and EMA target encoders.
   - Stage 2: load `checkpoints/stage1_energy.pt` only if needed for any sanity check; the diffusion model is built fresh and does not depend on stage 1.
   - Stage 3: load both `checkpoints/stage1_energy.pt` and `checkpoints/stage2_diffusion.pt`.
   - Stage 4: load `checkpoints/stage3_joint.pt`.
4. Cell to build the dataloader with the correct corruption mix for this stage (single-board for stage 0; easy mix for stages 1–3; hard mix for stage 4).
5. Cell to build models, optimizers, EMA trackers (both the JEPA target encoder tracker and the inference-EMA tracker via `diffusers.training_utils.EMAModel`), and the `accelerate.Accelerator` for mixed precision.
6. Markdown cell describing the loss for this stage.
7. Training loop cell (or a few cells: one epoch's worth of logic, plus the outer loop).
8. Cell to save the checkpoint to `checkpoints/`. Save the **full training state** (model state_dict, optimizer state, EMA target state, scheduler state, step count, RNG state) so any stage can be resumed cleanly.
9. Optional cells to print final metrics, sample a few outputs, run the legality gate (stage 2), etc.

What's inside the `inference/sample.py` notebook

1. Markdown cell describing the inference + evaluation flow.
2. Imports.
3. Load `checkpoints/stage4_final.pt`.
4. Build dataloader on the validation split (corruption is irrelevant here; just need problem and trace).
5. Cell defining the diffusion + energy refinement loop.
6. Cell defining the discretization (13-class softmax + metadata thresholding).
7. Cell running the loop over the validation set and accumulating predictions.
8. Cell computing exact-match and piece-wise accuracy.
9. Markdown cell summarizing results.

What a code-generating model should produce

When asked to "make stage 0" (or "make the VQ-VAE"), the model should produce:

- `core/vqvae.py` as a marimo notebook defining the `diffusers.VQModel` configuration and the chess-specific reconstruction loss.
- `core/dataset.py` if not already present (since stage 0 also needs the `BoardDataset`).
- `training/stage0_vqvae.py` as a marimo notebook that trains the VQ-VAE and writes `checkpoints/vqvae.pt`.

When asked to "make stage 1," the model should produce:

- Any new or updated `core/` notebooks needed (e.g., `core/encoder.py`, `core/energy.py`).
- The marimo notebook `training/stage1_energy.py` that imports from `core/`, loads the encoder weights from `checkpoints/vqvae.pt`, runs the training loop, and saves `checkpoints/stage1_energy.pt`.

When asked to "make the dataloader," the model should produce:

- `core/dataset.py` as a marimo notebook containing both `BoardDataset` (single boards, used by stage 0) and `ChessPairDataset` (3-tuples, used by stages 1–4), the three corruption strategies, the FEN↔tensor helpers, and sanity-check cells that exercise both datasets.

When asked to "make the inference loop," the model should produce:

- `inference/sample.py` as a marimo notebook that loads the final checkpoint, runs the diffusion + energy refinement loop, performs discretization, and computes evaluation metrics.

The marimo `.py` format itself is not optional or stylistic — every code file in this project (`core/`, `training/`, `inference/`) is a marimo notebook. The four folders (`core/`, `training/`, `inference/`, `checkpoints/`) are the entire project structure.

Board Representation

Every board is encoded as an 8×8×18 tensor of float values:

- **Channels 0–11:** piece occupancy. One channel per (piece type × color) combination: white pawn, white knight, white bishop, white rook, white queen, white king, then the same six for black. Each channel is a binary indicator of presence on each of the 64 squares. In raw clean boards these are one-hot across the 12 channels at any given square (or all zero for an empty square). During diffusion they take continuous values that get softmaxed at the end.
- **Channel 12:** side-to-move. Broadcast across all 64 squares: 1.0 if white to move, 0.0 if black to move.
- **Channels 13–16:** castling rights. Four channels broadcast across all 64 squares: white kingside, white queenside, black kingside, black queenside. 1.0 if available, 0.0 if not.
- **Channel 17:** en passant target square. A single channel with 1.0 at the target square if an en passant capture is currently legal, 0.0 elsewhere.

Halfmove and fullmove clocks are not encoded. The 50-move rule is out of scope.

The "empty square" state is implicit: it corresponds to all 12 piece channels being zero on that square. After softmax-at-the-end during inference, an empty-square logit must be added (effectively making it a 13-class softmax per square) — see the inference section.

Trace Definition

The trace board is the chess position after the single Stockfish-recommended best move has been applied to the problem board. It is one ply ahead. It is not a sequence, not a longer line of play, and not an embedding — it is a literal board state in the same 8×8×18 representation as the problem.

Dataset and Dataloader

The raw dataset is 180k pairs of FEN strings: (problem_fen, trace_fen). The dataloader emits 3-tuples:

```
(problem_board, clean_trace_board, corrupted_trace_board)
```

All three tensors are 8×8×18 float tensors.

The corrupted trace is generated on the fly per sample via one of three strategies. The sampling weights are configurable and change between training stages — see the Training Stages section. The three strategies are:

- **Batch shuffle.** Replace the trace with another random clean trace from the same batch. This produces a trace that is a valid chess position but unrelated to the problem. This is the easiest negative because the corrupted trace is uncorrelated with the problem.
- **Random legal move.** Apply a uniformly random legal move from the problem position (excluding the Stockfish best move) and use the resulting board. This is the hardest negative and forces the energy model to learn "best move" rather than just "legal position." Implementation requires a chess library (python-chess) at data loading time.
- **Random piece swap.** Take the clean trace and randomly swap two pieces, or move one piece to a random square. This produces a trace that is closely related to the correct one but not legally reachable. Medium difficulty.

If random-legal-move sampling fails (e.g., the only legal move is the best move, which can happen in forced positions), fall back to batch-shuffle for that sample.

The dataloader should be implemented once and used across all training stages, accepting the corruption mix as a constructor argument so the same code can drive both the easier early-stage split and the harder late-stage split. Batch size should target maximum GPU utilization on a 4070 Ti Super; start with 64 and tune up.

Two corruption mix presets are used in this project:

- **Easy mix (stages 1–3): 30% shuffle / 40% legal-move / 30% piece-swap.** Reduces the proportion of the hardest negatives so the energy model can find a working signal early, and gives the diffusion+energy joint training a stable energy landscape to operate against.
- **Hard mix (stage 4): 20% shuffle / 60% legal-move / 20% piece-swap.** Heavy on the hardest negatives. Forces the energy model to discriminate best-move from other-legal-move, which is the actual task the system needs to solve at inference time.

Architecture Components

Library Choice: HuggingFace Diffusers

The project uses HuggingFace's `diffusers` library wherever it gives a clean primitive. Specifically:

- `diffusers.VQModel` for the VQ-VAE in `core/vqvae.py`. After training, the encoder portion is extracted and wrapped for use in the energy model.
- `diffusers.UNet2DModel` for the diffusion U-Net in `core/diffusion.py`. We use the non-conditional U-Net (not `UNet2DConditionModel`) because conditioning is by channel concatenation rather than cross-attention — see the Diffusion Model section.
- `diffusers.DDPMScheduler` for the training noise schedule, and `diffusers.DDIMScheduler` for inference (deterministic sampling, fewer steps). Both expose the same API and are interchangeable; we use DDPM at training time and DDIM at inference.
- `diffusers.optimization.get_cosine_schedule_with_warmup` for the LR schedule.
- `accelerate.Accelerator` for mixed-precision training and gradient accumulation.

The energy model is hand-written — diffusers has no primitive for joint-embedding energy models. The energy-guided sampling loop is also hand-written, since it is the project's novel contribution.

Encoder (shared by both branches of the energy model)

A single encoder architecture is used for both the problem branch and the trace branch of the energy model. Weights are initialized from a VQ-VAE that is pretrained as part of this project (in `training/stage0_vqvae.py`, see the Training Stages section). The codebook and decoder of the VQ-VAE are discarded; only the encoder weights matter.

The encoder is the encoder portion of `diffusers.VQModel` instantiated with the architecture below (which matches the user's previously working configuration, adjusted for the project's 18-channel board representation):

```python
VQModel(
    sample_size=8,
    in_channels=18,
    out_channels=18,
    down_block_types=("DownEncoderBlock2D", "DownEncoderBlock2D"),
    up_block_types=("UpDecoderBlock2D", "UpDecoderBlock2D"),
    block_out_channels=(64, 128),
    layers_per_block=2,
    mid_block_add_attention=True,
    latent_channels=8,           # bumped from 4 → 8 for richer representation
    num_vq_embeddings=1024,
    norm_type="group",
    norm_num_groups=8,
    force_upcast=False,
    lookup_from_codebook=True,
)
```

This produces a **4×4×8 latent feature map** per board (8×8 input → one 2× downsample → 4×4 spatial; 8 latent channels). The bump from `latent_channels=4` to `latent_channels=8` is the only architectural change from the user's working configuration. With 180k training pairs, the slightly wider latent has plenty of data to support it, and the energy model's fusion head benefits from the extra capacity. Width (channel count) is preferred over depth here because the spatial resolution is already tiny.

Two encoder instances are maintained:

- **Problem encoder (online).** Initialized from VQ-VAE weights. The **second (last)** `DownEncoderBlock2D` is trainable; the first is frozen. This is a clean cut at the block boundary that matches the user's "first block frozen, last block trainable" intent.
- **Trace encoder (JEPA target).** Initialized from the same VQ-VAE weights. Its parameters are an exponential moving average of the problem encoder's parameters, updated after each gradient step:

```
θ_jepa_target ← τ_jepa · θ_jepa_target + (1 − τ_jepa) · θ_online
```

with τ_jepa = 0.999 as a starting point. This EMA is **structural**: without it, both encoders are identical and trainable, and the energy model trivially solves its loss by collapsing both branches to a constant. The trace encoder is never updated by gradient descent. This is the JEPA-style design that prevents representational collapse.

Both encoders process boards independently. The corrupted trace, when used, is passed through the *JEPA target* encoder (same as the clean trace).

This is a separate mechanism from the **inference EMA** described in the Project Conventions section, which maintains a slowly-moving snapshot of all trainable parameters (online encoder's last block, fusion head, diffusion U-Net) for use at evaluation time. The two EMAs run in parallel as independent readers of the live training weights, with different decay rates and different purposes.

Energy Model

The energy model takes a (problem_board, candidate_trace_board) pair and outputs a single scalar energy. Lower energy means "the candidate trace is a better continuation of the problem."

Architecture:

1. Encode problem with the online problem encoder → 4×4×8 feature map. Flatten to a 128-dim vector.
2. Encode candidate trace with the EMA target encoder → 4×4×8 feature map. Flatten to a 128-dim vector.
3. Fuse: concatenate `[e_p, e_t, e_p - e_t, e_p ⊙ e_t]` → 512-dim vector. The difference and elementwise product give the fusion head explicit "compare these two" signals.
4. Pass through a 3-layer MLP (512 → 256 → 128 → 1) with GELU activations and LayerNorm between layers → scalar.

The fusion head is fully trainable. The last block of the problem encoder is trainable. Everything else (target encoder, first block of the problem encoder) is not updated by gradient descent — though the target encoder still drifts via EMA.

Diffusion Model

A standard continuous denoising diffusion model that operates directly in 8×8×18 board space. No latent diffusion. Implemented with `diffusers.UNet2DModel`.

- **Input:** the noisy trace board x_t at timestep t (8×8×18), concatenated channel-wise with the clean problem board (8×8×18), giving an 8×8×36 input. The U-Net's `in_channels=36`, `out_channels=18`. The timestep is fed via `UNet2DModel`'s built-in timestep embedding.
- **Why channel concatenation only (no cross-attention):** chess move generation is a per-square reasoning task. Channel concatenation gives every conv kernel direct per-square access to both problem and trace at every layer, which is the strongest possible signal for "what changed between these boards." Since 60+ of 64 squares typically don't change between problem and trace, this aligned per-square view is the dominant useful signal. Cross-attention's strength is global content-addressing, but at 8×8 spatial size with two downsamples, the U-Net's bottleneck (2×2) already has effective full-board receptive field — there is no resolution-induced information bottleneck for cross-attention to fix. Reusing the energy model's encoder for cross-attention KV would also force that encoder to serve two objectives simultaneously (energy discrimination vs diffusion conditioning), and a separate diffusion-only problem-encoder would double encoder count. We therefore use channel concatenation exclusively, with no cross-attention layers anywhere in the U-Net.
- **Output:** the predicted clean board x̂_0. We use `prediction_type="sample"` in the scheduler to align with this.
- **Backbone configuration:** `UNet2DModel` with `block_out_channels=(128, 256, 256)`, `layers_per_block=2`, two downsamples (8→4→2), self-attention at the bottleneck (`mid_block_type="UNetMidBlock2D"` with attention enabled). Down block types: `("DownBlock2D", "AttnDownBlock2D")`; up block types: `("AttnUpBlock2D", "UpBlock2D")`. The widths are larger than the VQ-VAE's because the diffusion U-Net is doing more work and is the only large model trained from scratch.
- **Noise schedule:** `DDPMScheduler` with `num_train_timesteps=1000`, `beta_schedule="squaredcos_cap_v2"` (cosine schedule, the diffusers convention).
- **Inference scheduler:** `DDIMScheduler` constructed from the same config, with `num_inference_steps=50` (tunable). DDIM gives deterministic sampling, which makes energy-guided refinement more reproducible.

Inference Loop (Energy-Guided Diffusion)

The signature inference procedure interleaves diffusion and energy refinement. The energy gradient is computed with respect to the diffusion model's predicted clean board x̂_0 (per your decision in the design discussion). Pseudocode:

```
x_T ~ N(0, I)                              # initial noise, shape 8×8×18
for t in [T, T-1, ..., 1]:
    x_hat_0 = diffusion_model(x_t, problem, t)   # predicted clean board

    # Energy refinement: gradient descent on x_hat_0
    for k in range(K_refine):
        e = energy_model(problem, x_hat_0)
        g = ∂e / ∂x_hat_0
        x_hat_0 = x_hat_0 − η · g

    # Re-noise to x_{t-1} using the refined x_hat_0
    x_{t-1} = q_sample(x_hat_0, t-1) + diffusion_step_noise

final_board = softmax_per_square(x_hat_0_final)
```

Defaults to start: K_refine = 1, η = small (start with 0.01 and tune), 50 diffusion steps.

The energy gradient flows through the problem encoder and the fusion head, but since these are not being updated at inference time, only the gradient w.r.t. $\hat{x}_0$ is used.

Final Discretization

After the last diffusion+refinement step, the 12 piece channels at each square are augmented with a 13th "empty" logit (computed as a learned bias or simply 0.0 minus the max of the 12 piece logits — the cleanest choice is to add a learnable scalar bias for "empty") and softmaxed across the 13 categories. The argmax produces the discrete piece on each square.

The metadata channels (12–17) are thresholded at 0.5 to produce binary side-to-move, castling rights, and en passant.

If the final board is not a legal chess position (illegal piece counts, kings missing, etc.), the system simply returns it as-is and counts it as a failed prediction in evaluation. We do not project onto legality at inference time. The hope, supported by VQ-VAE reconstruction quality observed previously, is that legality emerges from training.

Training Stages

There are five stages. Stage 0 pretrains the VQ-VAE that supplies encoder weights to the energy model. Stages 1–3 use the **easy corruption mix (30/40/30)**. Stage 4 switches the dataloader to the **hard corruption mix (20/60/20)** to fine-tune the system on the harder discrimination problem the inference loop actually faces. The difficulty axis is the corruption mix on the dataloader; nothing else about the dataloader changes between stages 1 and 4.

Stage 0: VQ-VAE Pretraining

Goal: train a VQ-VAE that reconstructs 8×8×18 board tensors well enough that its encoder provides a useful initialization for the energy model.

Notebook: `training/stage0_vqvae.py`. This stage is run before any other stage. It is structurally similar to the user's existing VQ-VAE training script but adjusted for the 18-channel representation and to share the project's dataloader infrastructure.

Dataset: the same 180k pairs CSV. Stage 0 uses **only the problem FENs** (it is a pure reconstruction task; no need for trace pairs at this stage). The dataloader for stage 0 is a simpler `BoardDataset` that yields single boards rather than 3-tuples — this simpler dataset lives in `core/dataset.py` alongside the main paired dataset.

Architecture: the `diffusers.VQModel` configuration specified in the Encoder section above (4×4×8 latent, 1024 codebook entries, two ResNet down/up blocks at width 64/128).

Loss: a weighted combination, mirroring the user's existing approach.

- **Piece cross-entropy** on a 13-class softmax (12 piece classes + empty) per square. Weight: 1.0.
- **Piece MSE** as a stabilizer for early training. Weight: 0.25.
- **Metadata MSE** on channels 12–17 (side-to-move, castling, en passant). Weight: 0.5.
- **VQ commitment loss** as returned by `VQModel`. Weight: 1.0.

Trainable: all VQ-VAE parameters.

Output: `checkpoints/vqvae.pt`, containing the full VQ-VAE state. Stage 1 reads only the encoder weights and the `quant_conv` module from this checkpoint; the codebook, decoder, and `post_quant_conv` are loaded too (so the checkpoint is complete) but never used downstream.

Sanity checks at end of stage:

- Per-square piece accuracy on validation should exceed 95%. Below this, the encoder is not reconstructing reliably enough to be a useful initialization.
- Latent statistics (mean, std) are computed across the validation set and saved alongside the checkpoint, in case any downstream component needs to whiten latents.

Stage 1: Energy Model Pretraining (Easy Mix)

Goal: train the energy model on its own to assign low energy to (problem, clean_trace) and higher energy to (problem, corrupted_trace).

Dataloader: easy mix (30% shuffle / 40% legal-move / 30% piece-swap).

Loss: margin ranking loss.

```
L = max(0, m + E(problem, clean_trace) − E(problem, corrupted_trace))
```

Margin m starts at 1.0. Optionally add an InfoNCE-style listwise variant with multiple negatives per anchor — start with the simple pairwise version.

Trainable: fusion head, last block of online problem encoder. Target encoder updated via EMA. Diffusion model not yet involved.

This stage runs to convergence on the 180k training set. Monitor: average margin satisfaction rate (fraction of pairs where E_clean + m < E_corrupted) on a held-out validation split.

Stage 2: Diffusion Model Pretraining (Easy Mix)

Goal: train the diffusion model on its own to denoise clean traces given problem boards.

Dataloader: easy mix (the corrupted trace is unused in this stage but the dataloader emits it anyway; just ignore the third tuple element).

Loss: standard denoising loss. Sample a timestep t uniformly, add noise to the clean trace, predict x̂_0, compute MSE loss against the clean trace.

```
t ~ Uniform(1, T)
ε ~ N(0, I)
x_t = sqrt(ᾱ_t) · clean_trace + sqrt(1 − ᾱ_t) · ε
x̂_0 = diffusion_model(x_t, problem, t)
L = ||x̂_0 − clean_trace||²
```

Trainable: diffusion model only. Energy model is frozen and not used in this stage.

This stage establishes a baseline diffusion model that can produce plausible trace boards without any energy guidance.

End-of-stage legality gate. After stage 2 finishes, the stage 2 notebook runs a final sanity-check cell:

1. Sample 100 problems from the validation set.
2. For each, run the diffusion model end-to-end (no energy refinement) to produce a predicted trace.
3. Discretize via the standard 13-class softmax + argmax + metadata thresholding.
4. Convert each predicted board back to FEN and validate with `python-chess`'s `chess.Board(fen).is_valid()` — which checks piece counts, king presence, pawns on rank 1/8, and other structural rules.
5. Compute **legality rate** = fraction of predictions that are valid chess positions.

If legality rate < 50%, the cell renders a loud warning via `mo.callout(..., kind="danger")` recommending that the legality auxiliary loss be enabled in stage 3. This is a soft gate — the user decides whether to proceed.

Stage 3: Joint Training (Easy Mix)

Goal: continue training both models concurrently, each on its own loss, on the same batches. The energy model continues to refine its scoring; the diffusion model continues to refine its denoising.

Dataloader: easy mix.

For each batch:

1. Compute energy loss (margin ranking, as in Stage 1) and accumulate gradients into the energy model parameters.
2. Compute diffusion loss (denoising MSE, as in Stage 2, plus optionally the legality loss below) and accumulate gradients into the diffusion model parameters.
3. Step both optimizers.
4. Update the JEPA target encoder via its EMA (τ_jepa = 0.999).
5. Update the inference EMA on all trainable parameters (τ_inf = 0.9999).

There is no unrolled-loop backprop. The two networks share a dataloader and a training step but optimize independently. The energy model serves the diffusion model only at inference time, via guidance.

This stage is shorter than the first two — it is a refinement, not a from-scratch training. Length is empirical; monitor validation metrics and stop when they plateau.

Optional legality auxiliary loss (`use_legality_loss` flag). Disabled by default; enable if the stage 2 legality gate triggered a warning. When enabled, the diffusion loss adds a differentiable surrogate for chess legality, computed on the predicted $\hat{x}$_0:

```
L_legality = (
      α · piece_overcount_penalty       # > 1 piece on a single square
    + β · king_count_penalty            # ≠ 1 king of each color
    + γ · pawn_overcount_penalty        # > 8 pawns of either color
    + δ · pawn_promotion_rank_penalty   # any pawn channel mass on rank 1 or rank
8
)
```

Concretely (with $\hat{x}$_0 sigmoided to [0, 1] for the piece channels, since they are continuous during diffusion):

- **Piece overcount.** For each square, `relu(sum(piece_channels) - 1)²`, summed over 64 squares.
- **King count.** For each color, `(sum(king_channel) - 1)²`. Two terms.
- **Pawn overcount.** For each color, `relu(sum(pawn_channel) - 8)²`. Two terms.
- **Pawn promotion rank.** Sum of (white_pawn channel on rank 8) + (black_pawn channel on rank 1). Both are always illegal in any reachable position.

Suggested starting weights: $\alpha = 1.0$, $\beta = 1.0$, $\gamma = 0.5$, $\delta = 1.0$. The total legality loss is added to the diffusion MSE with a weight of 0.1 to start.

This is a non-exhaustive surrogate — it does not check check-legality, repetition, or castling-rights consistency. It is a coarse safety net, not a chess engine. The exact (non-differentiable) `python-chess` `is_valid()` check remains the evaluation-time legality validator.

Stage 4: Hard-Mix Fine-Tuning

Goal: fine-tune both models on the harder corruption mix to specialize the energy model on the best-move-vs-other-legal-moves discrimination, which is the de facto task at inference time.

Dataloader: hard mix (20% shuffle / 60% legal-move / 20% piece-swap). This is the *only* difference from Stage 3.

Training procedure: identical to Stage 3 — concurrent training of both models on their own losses, EMA update on target encoder, no unrolled-loop backprop. The legality loss flag carries through from stage 3 if it was enabled.

Considerations specific to this stage:

- **Lower learning rate.** Drop both optimizer learning rates by ~10× when entering this stage (e.g., 1e-4 → 1e-5). The harder negatives produce noisier gradients and the goal is refinement, not relearning.
- **Optional: re-warm-up the margin.** If the energy model's margin satisfaction rate collapses immediately on switching to the hard mix, that's expected. Watch validation trace-match rate, not just energy loss, as the stopping criterion.
- **Optional: anneal the mix.** If the abrupt jump from easy to hard is destabilizing, linearly interpolate the corruption weights over the first N steps of stage 4 from (30/40/30) to (20/60/20). This adds complexity to the dataloader; only adopt if needed.
- **Stopping.** When validation trace-match rate plateaus or regresses, stop. Stage 4 is short by design.

Evaluation

Single primary metric: **trace match rate** against Stockfish's best move on a held-out validation split.

For each (problem, true_trace) validation pair:

1. Run the full inference loop to produce a predicted trace board.
2. Compare the predicted board to the true trace board.

Two granularities of comparison:

- **Exact board match.** Predicted board equals true board on all 18 channels. This is the strict metric.
- **Piece-wise accuracy.** Fraction of the 64 squares where the predicted piece (after argmax) matches the true piece. This is the soft metric and is more useful during early training when exact matches are rare.

Track both during training and report exact-match rate as the headline number.

File / Module Layout

See the **Implementation Environment: Marimo Notebooks** section above for the authoritative file layout. The project uses four folders: `core/` for the dataloader and all model architectures (each as a marimo notebook), `training/` for the four training-stage notebooks, `inference/` for the inference and evaluation notebook, and `checkpoints/` for saved weights. Code is imported across notebooks using marimo's notebook-as-module support, which means `core/` notebooks must be definition-only (no top-level training or heavy side effects). Cross-stage state passes through checkpoint files on disk, not Python imports.

Key Hyperparameters and Defaults

Hyperparameter	Default	Notes
Board channels	18	12 pieces + 5 metadata + 1 en passant
Encoder embedding dim	match VQ-VAE	likely 256 or 512
EMA decay τ_jepa	0.999	for JEPA target encoder (structural, prevents collapse)
EMA decay τ_inf	0.9999	for inference-time EMA on all trainable params (sample quality)
Margin m	1.0	for stage 1 ranking loss
Corruption mix (stages 1–3)	30/40/30	shuffle / legal-move / piece-swap, easy
Corruption mix (stage 4)	20/60/20	shuffle / legal-move / piece-swap, hard
Diffusion timesteps (training)	1000	standard
Diffusion timesteps (inference)	50	tunable
Energy refinement steps per diffusion step	1	tunable, called K_refine
Energy refinement step size η	0.01	tune carefully
Batch size	64	start, scale up
Optimizer	AdamW	standard for both models
Learning rate	1e-4	start; decouple between models if needed

Project Conventions

These are global conventions shared across every notebook in the project.

Random seed

A single fixed seed of **42** is set at the top of every notebook. Seed Python's `random`, NumPy, and PyTorch (CPU + CUDA). CUDA determinism is *not* enforced — `torch.use_deterministic_algorithms(False)` and `torch.backends.cudnn.benchmark = True` for speed.

Train/val split

A single deterministic train/val split of **95% / 5%**, computed by hashing each row's index with a fixed seed and partitioning. Test set is not separated; the val set serves as both validation-during-training and final reporting. The split is computed once in `core/dataset.py` and reused identically across all stages.

Mixed precision

`accelerate.Accelerator(mixed_precision="bf16")` for all training. Inference runs in **fp32** for the energy refinement step specifically — the energy gradient signal is small and bf16 underflow can wipe it out. The diffusion forward pass at inference can stay in bf16; only the energy refinement needs the precision boost.

Inference-time EMA on trainable weights

In addition to the JEPA target encoder EMA (which is structural, prevents collapse, and runs at τ_jepa = 0.999), every stage maintains a second, parallel EMA of **all trainable parameters** for use at evaluation and as the deployed model. This includes:

- The trainable last block of the online problem encoder (stages 1+).
- The fusion head (stages 1+).
- The diffusion U-Net (stages 2+).

Decay rate **τ_inf = 0.9999** (slower than the JEPA EMA — this really is a long-term smoothed average). Implemented with `diffusers.training_utils.EMAModel` for the diffusion U-Net and a simple parameter-tracking utility for the energy model components.

Why both EMAs coexist cleanly:

- **JEPA target EMA** (τ = 0.999) is *consumed during training*: it provides the trace-side embeddings to the energy model on every forward pass.
- **Inference EMA** (τ = 0.9999) is *produced during training, consumed only at evaluation*: training itself uses the live (non-EMA) weights for the loss computation.

They are independent readers of the live training weights and never interact. Both are updated after each gradient step.

The inference notebook (`inference/sample.py`) loads the inference-EMA weights, not the live training weights, when reconstructing models from checkpoints. The checkpoint saver writes both `model_state_dict` (live) and `inference_ema_state_dict` (EMA) for every trainable component.

This is standard practice for diffusion training and consistently improves sample quality. It also stabilizes the energy landscape during stages 3–4 where both networks are being updated concurrently.

Wandb logging

Single wandb project named `chess-ebm-diffusion`. Each stage's notebook starts a wandb run with `name="stage{N}_{description}"` (e.g., `stage0_vqvae`, `stage1_energy`, `stage4_hard`). Every stage logs:

- Per-step training loss(es), broken out by component (energy, diffusion, legality if enabled).
- Per-epoch validation metrics.
- Per-epoch sample boards rendered as images via `display_board_tensors` and `wandb.Image`.
- Final checkpoint as a wandb artifact at the end of the stage.

If wandb is unavailable (no API key in env), notebooks should fall back to plain `print` + matplotlib rendering inline — no notebook should crash because wandb is missing.

Dataset format

The user's existing dataset is a **CSV file** with at minimum two columns: `problem_fen` and `trace_fen` (header names may differ in the actual file; `core/dataset.py` exposes column names as constructor arguments). The path to the CSV is configured via a single `DATA_CSV_PATH` constant defined in `core/dataset.py`. Stage 0 uses only the `problem_fen` column; stages 1–4 use both.

Checkpoint format

Every checkpoint is a single `.pt` file containing a dictionary with these keys (when applicable):

- `model_state_dict` — primary model weights, **live training values** (or for stage 3/4, a dict with `energy` and `diffusion` sub-keys).
- `inference_ema_state_dict` — EMA-smoothed version of the trainable parameters at $\tau_inf = 0.9999$, **used by the inference notebook**. Same nested structure as `model_state_dict` for stages 3/4.
- `jepa_target_encoder_state_dict` — JEPA target encoder weights ($\tau_jepa = 0.999$). Stages 1+.
- `optimizer_state_dict` — optimizer state (or per-model dict for stage 3/4).
- `scheduler_state_dict` — LR scheduler state.
- `step` — global training step.
- `epoch` — epoch number.
- `rng_state` — `{"python": ..., "numpy": ..., "torch": ..., "cuda": ...}` for clean resume.
- `config` — a snapshot of the stage's hyperparameter config dict, for reproducibility.

Stage 1 reads `model_state_dict` from `checkpoints/vqvae.pt`, slices out the encoder portion, and copies into both the online problem encoder and the JEPA target encoder. The inference EMA is initialized to match the online weights at the start of stage 1 and begins tracking from there. Subsequent stages always load whichever previous checkpoint is needed; resuming uses the live `model_state_dict` and continues the EMA trackers from `inference_ema_state_dict` and `jepa_target_encoder_state_dict`.

The inference notebook (`inference/sample.py`) **always loads from `inference_ema_state_dict`** for the deployed model, never from the live `model_state_dict`.

Critical Design Decisions Made

These are the answers from the design discussion that shaped the spec. A downstream code-generating model should treat these as fixed constraints unless explicitly told otherwise:

1. Continuous diffusion in raw board space, not latent space.
2. Diffusion predicts $\hat{x}_0$ (`prediction_type="sample"`), not noise.
3. Energy gradient is computed w.r.t. $\hat{x}_0$.
4. Single shared encoder architecture, with online (trainable last block) + JEPA EMA target variants.
5. Encoder weights initialized from a VQ-VAE pretrained in stage 0 of this project, decoder and codebook discarded.
6. The VQ-VAE uses `diffusers.VQModel` with the user's previously working configuration, with `latent_channels` bumped from 4 to 8 (4×4×8 latent feature map per board).
7. Diffusion is trained on real (problem, clean_trace) pairs with standard denoising loss, not on energy-model-generated traces.
8. The diffusion U-Net is `diffusers.UNet2DModel` with **channel-concatenation conditioning only** (in_channels=36, out_channels=18). No cross-attention layers in the U-Net at any level. The 8×8 spatial size + two downsamples gives sufficient receptive field that cross-attention adds complexity without a corresponding signal-quality gain for this domain.
9. **Two parallel EMA mechanisms** run during training:
   - **JEPA target encoder EMA** at $\tau = 0.999$, structural, used during training to produce trace-side embeddings.
   - **Inference EMA** at $\tau = 0.9999$, applied to all trainable parameters (online encoder's last block, fusion head, diffusion U-Net), used only at evaluation time and as the deployed model.
10. Stage 3 trains both networks concurrently on their own losses, no unrolled-loop backprop. Stage 4 repeats stage 3's procedure with a harder corruption mix and lower learning rate.
11. Legality is primarily learned, not enforced. A differentiable legality auxiliary loss is gated behind a `use_legality_loss` flag, enabled only if the stage 2 legality gate fires a warning. Inference does not project onto legality.
12. Final discretization is softmax across 13 classes (12 pieces + empty) per square; argmax for the final board.
13. Evaluation is exact match and piece-wise accuracy against the Stockfish best-move trace, computed using the **inference-EMA weights**, not the live training weights.
14. All notebooks use seed 42, 95/5 split, bf16 training, fp32 energy refinement, single wandb project `chess-ebm-diffusion`.
15. HuggingFace `diffusers` is used wherever a clean primitive exists (VQ-VAE, U-Net, schedulers, LR scheduler, EMAModel); the energy model and the energy-guided sampling loop are hand-written.

Open Items for Implementer Discretion

- Exact U-Net depth and width for the diffusion model — defaults specified in the Diffusion Model section, but VRAM-constrained tuning on the 4070 Ti Super is expected.
- Whether to use simple pairwise margin loss or InfoNCE in stage 1 (default: pairwise).
- Whether to share an optimizer or use separate optimizers in stage 3 (default: separate, one per model).
- Tuning of η, K_refine, and inference timestep count.
- Whether to add a small auxiliary reconstruction loss to keep the online encoder grounded during stage 1 (default: no).
- Exact weights of the legality loss components (α, β, γ, δ, and the overall multiplier) if the legality flag is enabled.
- Whether to anneal the corruption mix at the start of stage 4 (default: no, abrupt switch).

Out of Scope

- Multi-ply lookahead.
- Move legality projection / masking at inference.
- Stockfish eval scores as auxiliary supervision.
- 50-move rule / threefold repetition.
- Self-play, reinforcement learning, or online data generation.
- Curriculum learning over tactical difficulty (eval-score-based curriculum is out of scope; the only curriculum in this project is the corruption-mix transition between stages 3 and 4).